

```
import 'dart:async';
import 'dart:ui';
import 'package:flutter/material.dart';
import 'app.dart';
import 'services/app_state_service.dart';
import 'services/log_service.dart';
import 'theme/app_fonts.dart';
import 'theme/app_palette.dart';
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await LogService.instance.init();
  LogService.instance.info(LogTag.system, '应用启动');
  try {
    final state = await AppStateService().load();
    ThemeController.instance.switchTo(themeById(state.appTheme));
    FontScaleController.instance
      .switchTo(fontSizeLevelById(state.fontSizeLevel));
  } catch (e) {
    LogService.instance.error(LogTag.error, '恢复配色/字号失败, 使用默认', detail: '$e');
  }
  FlutterError.onError = (FlutterErrorDetails details) {
    LogService.instance.exception(
      'Flutter 框架异常',
      details.exception,
      details.stack,
    );
  };
  PlatformDispatcher.instance.onError = (error, stack) {
    LogService.instance.exception('平台/异步异常', error, stack);
    return true;
  };
  runApp(const EchoHymnApp());
}
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'screens/home_screen.dart';
import 'services/audio_service.dart';
import 'services/log_service.dart';
import 'theme/app_fonts.dart';
import 'theme/app_palette.dart';
import 'widgets/user_manual_dialog.dart';
final GlobalKey<NavigatorState> kNavigatorKey = GlobalKey<NavigatorState>();
class AppColors {
  static AppPalette get _p => ThemeController.instance.current;
  static Color get primary => _p.primary;
  static Color get primaryHover => _p.primaryHover;
  static Color get accent => _p.accent;
  static Color get pageBg => _p.pageBg;
  static Color get cardBg => _p.cardBg;
  static Color get titleBarBg => _p.titleBarBg;
```

```
static Color get topBarBg => _p.topBarBg;
static Color get sidebarBg => _p.sidebarBg;
static Color get rightPanelBg => _p.rightPanelBg;
static Color get versionBarBg => _p.versionBarBg;
static Color get playBarBg => _p.playBarBg;
static Color get statusBarBg => _p.statusBarBg;
static Color get controlBg => _p.controlBg;
static Color get controlBorder => _p.controlBorder;
static Color get textPrimary => _p.textPrimary;
static Color get textSecondary => _p.textSecondary;
static Color get textTertiary => _p.textTertiary;
static Color get divider => _p.divider;
static Color get border => _p.border;
static Color get success => _p.success;
static Color get warning => _p.warning;
static Color get danger => _p.danger;
static Color get selectedBg => _p.selectedBg;
static Color get lyricsBg => _p.lyricsBg;
static Color get scrollbarThumb => _p.scrollbarThumb;
static Color get windowBtnHover => _p.windowBtnHover;
}

KeyEventResult _handleGlobalShortcuts(FocusNode node, KeyEvent event) {
  if (event is! KeyDownEvent && event is! KeyRepeatEvent) {
    return KeyEventResult.ignored;
  }
  final bool isRepeat = event is KeyRepeatEvent;
  final audio = AudioService.instance;
  final logical = event.logicalKey;
  if (logical == LogicalKeyboardKey.mediaPlayPause) {
    if (!isRepeat) audio?.togglePlayPause();
    return KeyEventResult.handled;
  }
  if (logical == LogicalKeyboardKey.mediaTrackNext) {
    if (!isRepeat) audio?.playNext();
    return KeyEventResult.handled;
  }
  if (logical == LogicalKeyboardKey.mediaTrackPrevious) {
    if (!isRepeat) audio?.playPrev();
    return KeyEventResult.handled;
  }
  if (logical == LogicalKeyboardKey.audioVolumeUp) {
    audio?.changeVolume(0.05);
    return KeyEventResult.handled;
  }
  if (logical == LogicalKeyboardKey.audioVolumeDown) {
    audio?.changeVolume(-0.05);
    return KeyEventResult.handled;
  }
  if (logical == LogicalKeyboardKey.audioVolumeMute) {
    if (!isRepeat) audio?.toggleMute();
  }
}
```

```
        return KeyEventResult.handled;
    }
    if (logical == LogicalKeyboardKey.f1) {
        if (!isRepeat) {
            final navCtx = kNavigatorKey.currentContext;
            if (navCtx != null) UserManualDialog.show(navCtx);
        }
        return KeyEventResult.handled;
    }
    final keys = HardwareKeyboard.instance.logicalKeysPressed;
    final ctrl = keys.contains(LogicalKeyboardKey.controlLeft) ||
        keys.contains(LogicalKeyboardKey.controlRight);
    final alt = keys.contains(LogicalKeyboardKey.altLeft) ||
        keys.contains(LogicalKeyboardKey.altRight);
    final shift = keys.contains(LogicalKeyboardKey.shiftLeft) ||
        keys.contains(LogicalKeyboardKey.shiftRight);
    if (!ctrl && !alt && !shift) {
        if (logical == LogicalKeyboardKey.space) {
            if (isRepeat || !_isTextInputFocused()) return KeyEventResult.ignored;
            audio?.togglePlayPause();
            return KeyEventResult.handled;
        }
        return KeyEventResult.ignored;
    }
    if (ctrl && !alt && !shift) {
        if (logical == LogicalKeyboardKey.keyP) {
            if (!isRepeat) audio?.togglePlayPause();
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.arrowRight) {
            if (!isRepeat) audio?.playNext();
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.arrowLeft) {
            if (!isRepeat) audio?.playPrev();
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.arrowUp) {
            audio?.changeVolume(0.05);
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.arrowDown) {
            audio?.changeVolume(-0.05);
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.keyM) {
            if (!isRepeat) audio?.toggleMute();
            return KeyEventResult.handled;
        }
        return KeyEventResult.ignored;
    }
```

```

    }
    if (alt && !ctrl && !shift) {
        if (logical == LogicalKeyboardKey.arrowRight) {
            if (!isRepeat) audio?.playNext();
            return KeyEventResult.handled;
        }
        if (logical == LogicalKeyboardKey.arrowLeft) {
            if (!isRepeat) audio?.playPrev();
            return KeyEventResult.handled;
        }
        return KeyEventResult.ignored;
    }
    return KeyEventResult.ignored;
}

bool _isTextInputFocused() {
    final context = FocusManager.instance.primaryFocus?.context;
    if (context == null) return false;
    return context.findAncestorWidgetOfExactType<EditableText>() != null;
}

class EchoHymnApp extends StatelessWidget {
    const EchoHymnApp({super.key});
    @override
    Widget build(BuildContext context) {
        LogService.instance.info(LogTag.ui, '生成应用根组件 (EchoHymnApp / Material 主题)');
        return ValueListenableBuilder<FontSizeLevel>(
            valueListenable: FontScaleController.instance.notifier,
            builder: (context, level, _) {
                return ValueListenableBuilder<AppPalette>(
                    valueListenable: ThemeController.instance.notifier,
                    builder: (context, palette, _) {
                        final base = palette.brightness == Brightness.dark
                            ? ThemeData.dark(useMaterial3: true)
                            : ThemeData.light(useMaterial3: true);
                        final theme = base.copyWith(
                            colorScheme: ColorScheme.fromSeed(
                                seedColor: palette.primary,
                                brightness: palette.brightness,
                            ),
                            scaffoldBackgroundColor: palette.pageBg,
                            appBarTheme: AppBarTheme(
                                backgroundColor: palette.cardBg,
                                foregroundColor: palette.textPrimary,
                                elevation: 0.5,
                                titleTextStyle: TextStyle(
                                    color: palette.textPrimary,
                                    fontSize: 18,
                                    fontWeight: FontWeight.w600,
                                ),
                            ),
                            dividerColor: palette.divider,

```

```

textTheme: base.textTheme.apply(
  fontFamily: 'EchoSans',
  bodyColor: palette.textPrimary,
  displayColor: palette.textPrimary,
),
cardTheme: CardThemeData(
  color: palette.cardBg,
  elevation: 0,
  shape: const RoundedRectangleBorder(
    borderRadius: BorderRadius.all(Radius.circular(8)),
  ),
),
scrollbarTheme: ScrollbarThemeData(
  thickness: const WidgetStatePropertyAll(10),
  thumbColor: WidgetStatePropertyAll(palette.scrollbarThumb),
),
);
LogService.instance.info(
  LogTag.ui,
  'MaterialApp 构建完成（主题: ${palette.name}）',
  detail: '标题: EchoHymn · 聆听赞美诗\n 主页: HomeScreen',
);
return Focus(
  autofocus: true,
  onKeyEvent: _handleGlobalShortcuts,
  child: MaterialApp(
    title: 'EchoHymn · 聆听赞美诗',
    debugShowCheckedModeBanner: false,
    navigatorKey: kNavigatorKey,
    theme: theme,
    home: const HomeScreen(),
    builder: (context, child) {
      final s = AppFonts.scale;
      if (child == null) return const SizedBox.shrink();
      return LayoutBuilder(
        builder: (context, constraints) {
          final w = constraints.maxWidth;
          final h = constraints.maxHeight;
          return Align(
            alignment: Alignment.center,
            child: Transform.scale(
              scale: s,
              alignment: Alignment.center,
              child: SizedBox(
                width: w / s,
                height: h / s,
                child: child,
              ),
            ),
          );
        },
      );
    },
  );

```

```

        },
      );
    },
  ),
);
},
);
},
);
}
}
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import '../app.dart';
import '../models/hymn.dart';
import '../services/app_state_service.dart';
import '../services/audio_service.dart';
import '../services/chinese_convert_service.dart';
import '../services/log_service.dart';
import '../services/sqlite_repository.dart';
import '../theme/app_fonts.dart';
import '../theme/app_palette.dart';
import '../widgets/hymn_display.dart';
import '../widgets/panels/default_playlists_panel.dart';
import '../widgets/panels/hymn_list_panel.dart';
import '../widgets/panels/left_panel_base.dart';
import '../widgets/panels/my_playlists_panel.dart';
import '../widgets/user_manual_dialog.dart';
enum LeftTab { hymnList, defaultPlaylists, myPlaylists }
const MethodChannel _windowChannel = MethodChannel('echo_hymn/window');
const int kBaseWindowWidth = 850;
const int kBaseWindowHeight = 890;
const int kLeftPanelWidth = 350;
const int kRightPanelWidth = 600;
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
class _HomeScreenState extends State<HomeScreen> with WidgetsBindingObserver {
  SQLiteRepository? _repo;
  AudioService? _audio;
  bool _initError = false;
  final AppStateService _stateService = AppStateService.shared;
  bool _showLeft = false;
  bool _showRight = false;
  bool _windowMaximized = false;
  final GlobalKey _themeButtonKey = GlobalKey();

```

```

final GlobalKey _fontButtonKey = GlobalKey();
LeftTab _leftTab = LeftTab.hymnList;
int _restoredPlaylistIndex = -1;
List<Hymn> _allHymns = const [];
String? _playSubcategory;
String? _playPlaylistName;
String _currentAudioVersion = '钢琴版';
String _currentDisplayMode = 'lyrics';
AppState? _anchor;
@override
void initState() {
  super.initState();
  LogService.instance.info(LogTag.ui, 'HomeScreen 初始化 (initState)');
  WidgetsBinding.instance.addObserver(this);
  _windowChannel.setMethodCallHandler((call) async {
    if (call.method == 'onWindowMaximizedChanged') {
      final zoomed = call.arguments is bool && call.arguments as bool;
      if (mounted && zoomed != _windowMaximized) {
        setState(() => _windowMaximized = zoomed);
      }
    }
  });
  _init();
}
@override
void dispose() {
  _windowChannel.setMethodCallHandler(null);
  WidgetsBinding.instance.removeObserver(this);
  _saveState();
  _repo?.dispose();
  _audio?.dispose();
  LogService.instance.info(LogTag.system, 'HomeScreen 销毁 (应用关闭)');
  super.dispose();
}
@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.paused ||
      state == AppLifecycleState.inactive ||
      state == AppLifecycleState.detached) {
    LogService.instance.info(LogTag.system, '应用生命周期变化: ${state.name}, 保存状态');
    _saveState();
  }
}
Future<void> _init() async {
  try {
    LogService.instance.info(LogTag.ui, 'HomeScreen 初始化数据中...');
    final repo = await SqliteRepository.open();
    final audio = AudioService();
    await audio.loadSystemVolume();
    final state = await _stateService.load().catchError((_) => const AppState(

```

```

        leftTab: '',
        subcategory: '',
        playlistName: '',
        hymnNumber: '',
        audioVersion: '',
        displayMode: '',
        playlistIndex: -1,
    ));
    if (!mounted) return;
    audio.onCurrentChanged = () {
        _saveState();
        if (mounted) setState(() {});
    };
    setState(() {
        _repo = repo;
        _audio = audio;
        _allHymns = repo.getAllHymns();
        _currentAudioVersion =
            state.audioVersion.isEmpty ? '鋼琴版' : state.audioVersion;
        _currentDisplayMode =
            state.displayMode.isEmpty ? 'lyrics' : state.displayMode;
        _restoredPlaylistIndex = state.playlistIndex;
        _playSubcategory = state.subcategory.isEmpty ? null : state.subcategory;
        _playPlaylistName =
            state.playlistName.isEmpty ? null : state.playlistName;
        if (state.leftTab == 'defaultPlaylists') {
            _leftTab = LeftTab.defaultPlaylists;
        } else if (state.leftTab == 'myPlaylists') {
            _leftTab = LeftTab.myPlaylists;
        }
        _showLeft = state.showLeft;
        _showRight = state.showRight;
        ManualPrefs.instance.restoreFrom(state.manualOnStart);
    });
    _syncWindowSize();
    var restoreList = _allHymns;
    if (_playSubcategory != null) {
        final list = _hymnsOfSubcategory(_playSubcategory!);
        if (list.isNotEmpty) restoreList = list;
    } else if (_playPlaylistName != null) {
        final list = _hymnsOfPlaylistName(_playPlaylistName!);
        if (list.isNotEmpty) restoreList = list;
    }
    Hymn? hymn;
    if (restoreList.isNotEmpty) {
        hymn = state.hymnNumber.isNotEmpty
            ? _findInList(restoreList, state.hymnNumber)
            : restoreList.first;
    }
    if (hymn != null) {

```

```

        var idx = _restoredPlaylistIndex;
        if (idx < 0 || idx >= restoreList.length) {
            idx = restoreList.indexOf(hymn);
        }
        if (idx < 0) idx = 0;
        audio.setPlaylist(restoreList, startIndex: idx);
        audio.loadHymn(hymn, index: idx, version: _currentAudioVersion);
    }
    _anchor = AppState(
        leftTab: state.leftTab,
        subcategory: state.subcategory,
        playlistName: state.playlistName,
        hymnNumber: state.hymnNumber,
        audioVersion: _currentAudioVersion,
        displayMode: _currentDisplayMode,
        playlistIndex: _restoredPlaylistIndex,
    );
    setState(() {});
    _syncWindowAppearance();
    LogService.instance.info(
        LogTag.ui,
        'HomeScreen 初始化完成',
        detail: '诗歌总数: ${_allHymns.length}',
    );
    LogService.instance.info(LogTag.ui, 'HomeScreen 首帧构建完成');
    if (ManualPrefs.instance.showOnStart) {
        WidgetsBinding.instance.addPostFrameCallback((_) {
            if (mounted) {
                LogService.instance.info(LogTag.ui, '启动自动弹出用户手册');
                UserManualDialog.show(context);
            }
        });
    }
} catch (e) {
    if (!mounted) return;
    LogService.instance.error(LogTag.error, 'HomeScreen 初始化失败', detail: '$e');
    setState(() => _initError = true);
}
}

Hymn? _findInList(List<Hymn> list, String number) {
    for (final h in list) {
        if (h.hymnNumber == number) return h;
    }
    return null;
}

List<Hymn> _hymnsOfSubcategory(String subcategory) {
    final cats = _repo?.getAllCategories() ?? const [];
    final list = <Hymn>[];
    for (final c in cats) {
        if (c.subcategory == subcategory) {

```

```

        for (final e in c.hymns) {
            final h = _repo?.hymnByNumber(e.value.toString());
            if (h != null) list.add(h);
        }
    }
}
return list;
}
List<Hymn> _hymnsOfPlaylistName(String name) {
    final pls = _repo?.getPlaylists() ?? const [];
    final list = <Hymn>[];
    for (final pl in pls) {
        if (pl.name == name) {
            for (final item in pl.hymns) {
                final h = _repo?.hymnByNumber(item.value.toString());
                if (h != null) list.add(h);
            }
        }
    }
    return list;
}
Future<void> _saveState() async {
    final hymn = _audio?.currentHymn;
    var leftTab = _leftTab.name;
    final list = _audio?.playlist ?? const <Hymn>[];
    var saveIndex = false;
    if (list.isNotEmpty) {
        if (_playSubcategory != null) {
            leftTab = LeftTab.defaultPlaylists.name;
        } else if (_playPlaylistName != null) {
            leftTab = LeftTab.myPlaylists.name;
        } else {
            leftTab = LeftTab.hymnList.name;
            saveIndex = _containsAllHymns(list);
        }
    }
    await _stateService.saveAll(
        leftTab: leftTab,
        subcategory: _playSubcategory ?? '',
        playlistName: _playPlaylistName ?? '',
        hymnNumber: hymn?.hymnNumber ?? '',
        audioVersion: _currentAudioVersion,
        displayMode: _currentDisplayMode,
        playlistIndex: saveIndex ? (_audio?.currentIndex ?? -1) : -1,
        showLeft: _showLeft,
        showRight: _showRight,
        appTheme: ThemeController.instance.current.id,
        fontSizeLevel: FontScaleController.instance.current.id,
        manualOnStart: ManualPrefs.instance.showOnStart,
    );
}

```

```

}
bool _containsAllHymns(List<Hymn> list) {
  if (list.length != _allHymns.length) return false;
  for (var i = 0; i < list.length; i++) {
    if (list[i].id != _allHymns[i].id) return false;
  }
  return true;
}
Future<void> _syncWindowSize() async {
  if (kIsWeb) return;
  try {
    final s = AppFonts.scale;
    await _windowChannel.invokeMethod<void>('setClientSize', {
      'width': kBaseWindowWidth,
      'height': kBaseWindowHeight,
      'leftPanelWidth': _showLeft ? (kLeftPanelWidth * s).round() : 0,
      'rightPanelWidth': _showRight ? kRightPanelWidth : 0,
    });
    if (mounted) setState(() {});
  } catch (_) {}
}
@override
Widget build(BuildContext context) {
  return ValueListenableBuilder<FontSizeLevel>(
    valueListenable: FontScaleController.instance.notifier,
    builder: (context, __, ___) {
      return ValueListenableBuilder<AppPalette>(
        valueListenable: ThemeController.instance.notifier,
        builder: (context, __, ___) {
          return Scaffold(
            body: Column(
              children: [
                _buildTitleBar(),
                Divider(height: 1, color: AppColors.divider),
                _buildTopBar(),
                Expanded(child: _buildBody()),
                Divider(height: 1, color: AppColors.divider),
                _buildStatusBar(),
              ],
            ),
          );
        },
      );
    },
  );
}
Widget _buildTitleBar() {
  return Container(
    height: 30,

```

```

color: AppColors.titleBarBg,
child: Stack(
  children: [
    Positioned.fill(
      child: GestureDetector(
        behavior: HitTestBehavior.opaque,
        onDoubleTap: _toggleMaximize,
        onPanStart: (_) => _startWindowDrag(),
      ),
    ),
  ],
  Align(
    alignment: Alignment.centerLeft,
    child: Padding(
      padding: const EdgeInsets.only(left: 12),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Container(
            width: 22,
            height: 22,
            decoration: BoxDecoration(
              color: AppColors.primary,
              borderRadius: BorderRadius.circular(6),
            ),
            child: const Icon(Icons.music_note,
              size: 14, color: Colors.white),
          ),
          const SizedBox(width: 8),
          Text(
            'EchoHymn · 聆听赞美诗',
            style: TextStyle(
              fontSize: 13,
              fontWeight: FontWeight.w600,
              color: AppColors.textPrimary,
            ),
          ),
        ],
      ),
    ),
  ),
  Align(
    alignment: Alignment.centerRight,
    child: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        _WindowButton(
          key: _themeButtonKey,
          icon: Icons.palette_outlined,
          tooltip: '切换配色（${ThemeController.instance.current.name}）',
          onTap: _showThemeMenu,

```

```

    ),
    _WindowButton(
      key: _fontButtonKey,
      icon: Icons.format_size,
      tooltip:
        '切换字号（${FontScaleController.instance.current.label}）',
      onTap: _showFontSizeMenu,
    ),
    _WindowButton(
      icon: Icons.help_outline,
      tooltip: '用户手册（F1）',
      onTap: _showManual,
    ),
    _WindowButton(
      icon: Icons.minimize,
      tooltip: '最小化',
      onTap: _minimizeWindow,
    ),
    _WindowButton(
      icon:
        _windowMaximized ? Icons.filter_none : Icons.crop_square,
      tooltip: _windowMaximized ? '还原' : '最大化',
      onTap: _toggleMaximize,
    ),
    _WindowButton(
      icon: Icons.close,
      tooltip: '关闭',
      danger: true,
      onTap: _closeWindow,
    ),
  ],
),
),
],
),
);
}

Widget _buildTopBar() {
  final hymn = _audio?.currentHymn;
  final centerText = hymn != null
    ? '第 ${hymn.hymnNumber} 首《'
      '${ChineseConvertService.instance.toSimplified(hymn.title)}》'
    : 'EchoHymn · 聆听赞美诗';
  return Container(
    height: 40,
    color: AppColors.topBarBg,
    child: Row(
      children: [
        SizedBox(
          width: 60,

```

```

        height: 30,
        child: _toggleButton(
          icon: _showLeft ? Icons.chevron_right : Icons.chevron_left,
          tooltip: _showLeft ? '收起左侧栏目' : '展开左侧栏目',
          active: !_showLeft,
          onTap: () {
            LogService.instance.info(
              LogTag.action,
              _showLeft ? '收起左侧栏目' : '展开左侧栏目',
            );
            setState(() => _showLeft = !_showLeft);
            _syncWindowSize();
            _saveState();
          },
        ),
      ),
    Expanded(
      child: Center(
        child: Text(
          centerText,
          style: TextStyle(
            fontSize: 15,
            fontWeight: FontWeight.w600,
            color: AppColors.textPrimary,
          ),
          overflow: TextOverflow.ellipsis,
        ),
      ),
    ),
    SizedBox(
      width: 60,
      height: 30,
      child: _toggleButton(
        icon: _showRight ? Icons.chevron_left : Icons.chevron_right,
        tooltip: _showRight ? '收起右侧栏目' : '展开右侧栏目',
        active: !_showRight,
        onTap: () {
          LogService.instance.info(
            LogTag.action,
            _showRight ? '收起右侧栏目' : '展开右侧栏目',
          );
          setState(() => _showRight = !_showRight);
          _syncWindowSize();
          _saveState();
        },
      ),
    ),
  ],
),
);

```

```

}
Widget _toggleButton({
  required IconData icon,
  required String tooltip,
  required bool active,
  required VoidCallback onTap,
}) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 4),
    child: Tooltip(
      message: tooltip,
      child: Material(
        color: active ? AppColors.primary : AppColors.controlBg,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(6),
          side: BorderSide(
            color: active ? Colors.transparent : AppColors.controlBorder,
            width: 1,
          ),
        ),
        child: InkWell(
          borderRadius: BorderRadius.circular(6),
          onTap: onTap,
          child: Center(
            child: Icon(
              icon,
              size: 18,
              color: active ? Colors.white : AppColors.textSecondary,
            ),
          ),
        ),
      ),
    ),
  );
}

void _showManual() {
  LogService.instance.info(LogTag.action, '打开用户手册');
  UserManualDialog.show(context);
}

Future<void> _showThemeMenu() async {
  final overlay =
    Overlay.of(context).context.findRenderObject() as RenderBox?;
  final btnBox =
    _themeButtonKey.currentContext?.findRenderObject() as RenderBox?;
  if (overlay == null || btnBox == null) return;
  final current = ThemeController.instance.current;
  final chosen = await showMenu<String>(
    context: context,
    position: RelativeRect.fromRect(
      Rect.fromPoints(

```

```

        btnBox.localToGlobal(Offset.zero, ancestor: overlay),
        btnBox.localToGlobal(btnBox.size.bottomRight(Offset.zero),
            ancestor: overlay),
    ),
    Offset.zero & overlay.size,
),
items: kThemes.map((t) {
    return PopupMenuItem<String>(
        value: t.id,
        height: 36,
        child: Row(
            children: [
                Container(
                    width: 14,
                    height: 14,
                    decoration: BoxDecoration(
                        color: t.primary,
                        shape: BoxShape.circle,
                    ),
                ),
                const SizedBox(width: 10),
                Text(
                    t.name,
                    style:
                        const TextStyle(fontSize: 13, fontWeight: FontWeight.w500),
                ),
                const Spacer(),
                if (t.id == current.id)
                    Icon(Icons.check, size: 16, color: AppColors.textSecondary),
            ],
        ),
    );
}).toList(),
);
if (chosen != null && mounted) _switchTheme(themeById(chosen));
}
void _switchTheme(AppPalette palette) {
    if (palette.id == ThemeController.instance.current.id) return;
    LogService.instance.info(LogTag.action, '切换配色: ${palette.name}');
    ThemeController.instance.switchTo(palette);
    _syncWindowAppearance();
    _saveState();
}
Future<void> _showFontSizeMenu() async {
    final overlay =
        Overlay.of(context).context.findRenderObject() as RenderBox?;
    final btnBox =
        _fontButtonKey.currentContext?.findRenderObject() as RenderBox?;
    if (overlay == null || btnBox == null) return;
    final current = FontScaleController.instance.current;

```

```

final chosen = await showMenu<FontSizeLevel>(
  context: context,
  position: RelativeRect.fromRect(
    Rect.fromPoints(
      btnBox.localToGlobal(Offset.zero, ancestor: overlay),
      btnBox.localToGlobal(btnBox.size.bottomRight(Offset.zero),
        ancestor: overlay),
    ),
    Offset.zero & overlay.size,
  ),
  items: FontSizeLevel.values.map((l) {
    return PopupMenuItem<FontSizeLevel>(
      value: l,
      height: 36,
      child: Row(
        children: [
          Icon(
            l == FontSizeLevel.normal
              ? Icons.format_size
              : Icons.text_fields,
            size: 16,
            color: AppColors.textSecondary,
          ),
          const SizedBox(width: 10),
          Text(l.label, style: const TextStyle(fontSize: 13)),
          const Spacer(),
          if (l == current)
            Icon(Icons.check, size: 16, color: AppColors.textSecondary),
        ],
      ),
    );
  }).toList(),
);
if (chosen != null && mounted) _switchFontSize(chosen);
}

void _switchFontSize(FontSizeLevel level) {
  if (level == FontScaleController.instance.current) return;
  FontScaleController.instance.switchTo(level);
  LogService.instance.info(
    LogTag.action,
    '切换字号: ${level.label} (×${level.scale}) ',
  );
  if (_showLeft || _showRight) _syncWindowSize();
  _saveState();
}

Future<void> _syncWindowAppearance() async {
  try {
    final c = AppColors.titleBarBg;
    final isDark =
      ThemeController.instance.current.brightness == Brightness.dark;

```

```

    LogService.instance.info(
      LogTag.action,
      '同步窗口外观',
      detail: '主题: ${ThemeController.instance.current.name} / isDark: '
        '$isDark / 边框色: #'
        '${c.r.toInt().toRadixString(16).padLeft(2, '0')}'
        '${c.g.toInt().toRadixString(16).padLeft(2, '0')}'
        '${c.b.toInt().toRadixString(16).padLeft(2, '0')}',
    );
    await _windowChannel.invokeMethod<void>('setWindowAppearance', {
      'isDark': isDark,
      'borderColor': (c.r.toInt() << 16) | (c.g.toInt() << 8) | c.b.toInt(),
    });
  } catch (e) {
    LogService.instance.error(LogTag.error, '同步窗口外观失败', detail: '$e');
  }
}

Future<void> _minimizeWindow() async {
  LogService.instance.info(LogTag.action, '窗口最小化');
  try {
    await _windowChannel.invokeMethod<void>('minimize');
  } catch (_) {
  }
}

Future<void> _toggleMaximize() async {
  LogService.instance
    .info(LogTag.action, _windowMaximized ? '窗口还原' : '窗口最大化');
  try {
    await _windowChannel.invokeMethod<void>('maximizeToggle');
  } catch (_) {
  }
}

Future<void> _closeWindow() async {
  LogService.instance.info(LogTag.action, '点击关闭按钮');
  try {
    await _windowChannel.invokeMethod<void>('close');
  } catch (_) {
  }
}

Future<void> _startWindowDrag() async {
  try {
    await _windowChannel.invokeMethod<void>('startWindowDrag');
  } catch (_) {
  }
}

Widget _buildBody() {
  if (_initError) {
    return const Center(child: Text('数据加载失败, 请检查 data/tjc_hymn.db'));
  }
  if (_repo == null || _audio == null) {

```

```

        return const Center(child: CircularProgressIndicator());
    }
    final repo = _repo!;
    final audio = _audio!;
    return Row(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        if (_showLeft) ...[
          _buildLeftPanel(repo, audio),
          VerticalDivider(width: 1, color: AppColors.divider),
        ],
        Expanded(
          child: HymnDisplay(
            audio: audio,
            initialMode: _currentDisplayMode,
            onModeChanged: (mode) {
              _currentDisplayMode = mode;
              _saveState();
            },
            onAudioVersionChanged: (v) {
              _currentAudioVersion = v;
              _saveState();
            },
          ),
        ),
        if (_showRight) ...[
          VerticalDivider(width: 1, color: AppColors.divider),
          _buildRightPanel(),
        ],
      ],
    );
}

Widget _buildLeftPanel(SqliteRepository repo, AudioService audio) {
  final leftWidth = kLeftPanelWidth / MediaQuery.devicePixelRatioOf(context);
  return SizedBox(
    width: leftWidth,
    child: Container(
      color: AppColors.sidebarBg,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          _buildLeftTabBar(),
          Divider(height: 1, color: AppColors.divider),
          Expanded(
            child: switch (_leftTab) {
              LeftTab.hymnList => HymnListPanel(
                key: const ValueKey('hymnList'),
                audio: audio,
                repo: repo,
                anchor: _anchor,

```

```

        onPlayback: _onPlayback,
      ),
      LeftTab.defaultPlaylists => DefaultPlaylistsPanel(
        key: const ValueKey('defaultPlaylists'),
        audio: audio,
        repo: repo,
        anchor: _anchor,
        onPlayback: _onPlayback,
      ),
      LeftTab.myPlaylists => MyPlaylistsPanel(
        key: const ValueKey('myPlaylists'),
        audio: audio,
        repo: repo,
        anchor: _anchor,
        onPlayback: _onPlayback,
      ),
    ],
  ),
);
}

Widget _buildLeftTabBar() {
  return Padding(
    padding: const EdgeInsets.all(8),
    child: Row(
      children: [
        _tabButton('诗歌列表', LeftTab.hymnList, Icons.list),
        const SizedBox(width: 4),
        _tabButton('默认歌单', LeftTab.defaultPlaylists, Icons.library_music),
        const SizedBox(width: 4),
        _tabButton('个人歌单', LeftTab.myPlaylists, Icons.favorite),
      ],
    ),
  );
}

Widget _tabButton(String label, LeftTab tab, IconData icon) {
  final selected = _leftTab == tab;
  return Expanded(
    child: Material(
      color: selected ? AppColors.selectedBg : AppColors.sidebarBg,
      borderRadius: BorderRadius.circular(6),
      child: InkWell(
        borderRadius: BorderRadius.circular(6),
        onTap: () {
          LogService.instance.info(
            LogTag.action,
            '切换左侧栏目: $label',
          );
        },
      ),
    ),
  );
}

```

```

        setState(() => _leftTab = tab);
        _saveState();
    },
    child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 6),
        child: Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
                Icon(icon,
                    size: 16,
                    color:
                        selected ? AppColors.primary : AppColors.textSecondary),
                const SizedBox(width: 4),
                Text(
                    label,
                    style: TextStyle(
                        fontSize: 13,
                        fontWeight: selected ? FontWeight.w600 : FontWeight.normal,
                        color:
                            selected ? AppColors.primary : AppColors.textSecondary,
                    ),
                ),
            ],
        ),
    ),
),
),
);
}
void _onPlayback(PlaybackEvent e) {
    LogService.instance.info(
        LogTag.play,
        '面板点击播放: 第 ${e.hymn.hymnNumber} 首 《${e.hymn.title}》 ',
        detail: '来源子目录: ${e.sourceSubcategory ?? '无'}\n'
            '来源个人歌单: ${e.sourcePlaylistName ?? '无'}\n'
            '列表位置: ${e.index}',
    );
    _playSubcategory = e.sourceSubcategory;
    _playPlaylistName = e.sourcePlaylistName;
    if (e.sourceSubcategory != null) {
        _leftTab = LeftTab.defaultPlaylists;
    } else if (e.sourcePlaylistName != null) {
        _leftTab = LeftTab.myPlaylists;
    } else {
        _leftTab = LeftTab.hymnList;
    }
    setState(() {});
    _saveState();
}
Widget _buildRightPanel() {

```

```

final hymn = _audio?.currentHymn;
final rightWidth = kRightPanelWidth /
    AppFonts.scale /
    MediaQuery.devicePixelRatioOf(context);
return SizedBox(
  width: rightWidth,
  child: Container(
    color: AppColors.rightPanelBg,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(10),
          child: Text(
            '诗歌源考',
            style: TextStyle(
              fontSize: 14,
              fontWeight: FontWeight.w600,
              color: AppColors.textPrimary,
            ),
          ),
        ),
        Divider(height: 1, color: AppColors.divider),
        Expanded(
          child: hymn == null
            ? const _EmptyHint(text: '请选择一首诗歌')
            : SingleChildScrollView(
                padding: const EdgeInsets.all(16),
                child: Text(
                  ChineseConvertService.instance.toSimplified(
                    hymn.sourceInfo.isEmpty
                      ? '暂无源考资料'
                      : hymn.sourceInfo),
                  style: TextStyle(
                    fontSize: 13,
                    height: 1.6,
                    color: AppColors.textSecondary,
                  ),
                ),
              ),
        ),
      ],
    ),
  );
}

Widget _buildStatusBar() {
  final hymn = _audio?.currentHymn;
  return Container(
    height: 30,

```

```

color: AppColors.statusBarBg,
child: Row(
  children: [
    const SizedBox(width: 16),
    if (hymn != null)
      Expanded(
        child: Text(
          ChineseConvertService.instance.toSimplified(hymn.statusMeta),
          style: TextStyle(fontSize: 12, color: AppColors.textSecondary),
          overflow: TextOverflow.ellipsis,
        ),
      )
    else
      Expanded(
        child: Text(
          '就绪',
          style: TextStyle(fontSize: 12, color: AppColors.textTertiary),
        ),
      ),
    StreamBuilder<Duration>(
      stream: _audio?.positionStream ?? const Stream<Duration>.empty(),
      builder: (context, posSnap) {
        final pos = posSnap.data ?? Duration.zero;
        return StreamBuilder<Duration>(
          stream:
            _audio?.durationStream ?? const Stream<Duration>.empty(),
          builder: (context, durSnap) {
            final dur = durSnap.data ?? Duration.zero;
            final pct = dur.inMilliseconds > 0
              ? (pos.inMilliseconds * 100 / dur.inMilliseconds).floor()
              : 0;
            return Padding(
              padding: const EdgeInsets.only(right: 16),
              child: Text(
                '${formatTime(pos)} / ${formatTime(dur)} · $pct%',
                style: TextStyle(
                  fontSize: 12, color: AppColors.textSecondary),
              ),
            );
          },
        );
      },
    ),
  ],
),
);
}
}
class _WindowButton extends StatefulWidget {
  final IconData icon;

```

```

final String tooltip;
final VoidCallback onTap;
final bool danger;
const _WindowButton({
  super.key,
  required this.icon,
  required this.tooltip,
  required this.onTap,
  this.danger = false,
});
@override
State<_WindowButton> createState() => _WindowButtonState();
}
class _WindowButtonState extends State<_WindowButton> {
  bool _hover = false;
  @override
  Widget build(BuildContext context) {
    return MouseRegion(
      onEnter: (_) => setState(() => _hover = true),
      onExit: (_) => setState(() => _hover = false),
      child: SizedBox(
        width: 42,
        height: 30,
        child: Tooltip(
          message: widget.tooltip,
          child: InkWell(
            onTap: widget.onTap,
            child: Container(
              color: _hover
                ? (widget.danger
                  ? const Color(0xFFE81123)
                  : AppColors.windowBtnHover)
                : Colors.transparent,
              child: Icon(
                widget.icon,
                size: 16,
                color: _hover && widget.danger
                  ? Colors.white
                  : AppColors.textSecondary,
              ),
            ),
          ),
        ),
      );
  }
}
class _EmptyHint extends StatelessWidget {
  final String text;
  const _EmptyHint({required this.text});

```

```

@override
Widget build(BuildContext context) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Icon(Icons.music_off, size: 48, color: AppColors.textTertiary),
        const SizedBox(height: 12),
        Text(text,
          style: TextStyle(fontSize: 14, color: AppColors.textSecondary)),
      ],
    ),
  );
}

import 'dart:async';
import 'dart:io';
import 'dart:math' as math;
import 'package:flutter/gestures.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart' show HardwareKeyboard;
import '../app.dart';
import '../models/hymn.dart';
import '../services/audio_service.dart';
import '../services/app_paths.dart';
import '../services/chinese_convert_service.dart';
import '../services/log_service.dart';
import '../theme/app_fonts.dart';
enum DisplayMode { lyrics, numbered, staff }
class HymnDisplay extends StatefulWidget {
  final AudioService audio;
  final String? initialMode;
  final ValueChanged<String>? onModeChanged;
  final ValueChanged<String>? onAudioVersionChanged;
  const HymnDisplay({
    super.key,
    required this.audio,
    this.initialMode,
    this.onModeChanged,
    this.onAudioVersionChanged,
  });
  @override
  State<HymnDisplay> createState() => _HymnDisplayState();
}
class _HymnDisplayState extends State<HymnDisplay> {
  late DisplayMode _mode;
  StreamSubscription? _statusSub;
  @override
  void initState() {
    super.initState();

```

```

LogService.instance.info(LogTag.ui, '主内容区 HymnDisplay 初始化');
_mode = DisplayMode.values.firstWhere(
  (m) => m.name == widget.initialMode,
  orElse: () => DisplayMode.lyrics,
);
_statusSub = widget.audio.statusStream.listen((s) {
  if (!mounted) return;
  setState(() {});
  if (s == PlayerStatus.error) _showAudioError();
});
}
@override
void dispose() {
  _statusSub?.cancel();
  super.dispose();
}
String get currentModeName => _mode.name;
@override
Widget build(BuildContext context) {
  final hymn = widget.audio.currentHymn;
  return Container(
    color: AppColors.pageBg,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        _buildVersionBar(hymn),
        Divider(height: 1, color: AppColors.divider),
        Expanded(child: _buildContent(hymn)),
        Divider(height: 1, color: AppColors.divider),
        _buildPlayerBar(hymn),
      ],
    ),
  );
}
Widget _buildVersionBar(Hymn? hymn) {
  return Container(
    height: 44,
    color: AppColors.versionBarBg,
    padding: const EdgeInsets.symmetric(horizontal: 12),
    child: Row(
      children: [
        _versionBtn(
          label: '钢琴版',
          icon: Icons.piano,
          active:
            hymn != null && widget.audio.currentAudioVersion.contains('钢琴'),
          onTap: hymn == null ? null : () => _switchVersion(hymn, '钢琴版'),
        ),
        const SizedBox(width: 8),
        _buildVoiceButton(hymn),
      ],
    ),
  );
}

```

```

        const Spacer(),
        _modeBtn('歌词', DisplayMode.lyrics, Icons.lyrics_outlined),
        const SizedBox(width: 4),
        _modeBtn('简谱', DisplayMode.numbered, Icons.music_note),
        const SizedBox(width: 4),
        _modeBtn('五线谱', DisplayMode.staff, Icons.graphic_eq),
      ],
    ),
  );
}
Widget _versionBtn({
  required String label,
  required IconData icon,
  required bool active,
  VoidCallback? onTap,
}) {
  return Material(
    color: active ? AppColors.primary : AppColors.controlBg,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(6),
      side: BorderSide(
        color: active ? Colors.transparent : AppColors.controlBorder,
        width: 1,
      ),
    ),
    child: InkWell(
      borderRadius: BorderRadius.circular(6),
      onTap: onTap,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        child: Row(
          children: [
            Icon(icon,
              size: 16,
              color: active ? Colors.white : AppColors.textSecondary),
            const SizedBox(width: 4),
            Text(
              label,
              style: TextStyle(
                fontSize: 13,
                color: active ? Colors.white : AppColors.textSecondary,
              ),
            ),
          ],
        ),
      ),
    ),
  );
}
List<String> voiceVersions(Hymn hymn) {

```

```

    final all = widget.audio.availableVersions(hymn);
    return all.where((v) => !v.contains('鋼琴')).toList();
}
String _voiceLabel(String v) =>
    ChineseConvertService.instance.toSimplified(v);
Widget _buildVoiceButton(Hymn? hymn) {
    final voices = hymn == null ? <String>[] : voiceVersions(hymn);
    if (hymn == null || voices.isEmpty) {
        return _versionBtn(label: '人声版', icon: Icons.mic, active: false);
    }
    final currentVoice = widget.audio.currentAudioVersion;
    final isVoiceActive = currentVoice.contains('人聲') ||
        currentVoice.contains('人声') ||
        !currentVoice.contains('鋼琴');
    return _versionBtn(
        label: '人声版',
        icon: Icons.mic,
        active: isVoiceActive,
        onTap: () => _switchVersion(hymn, voices.first),
    );
}
Widget _modeBtn(String label, DisplayMode mode, IconData icon) {
    final selected = _mode == mode;
    return Material(
        color: selected ? AppColors.selectedBg : AppColors.controlBg,
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(6),
            side: BorderSide(
                color: selected ? Colors.transparent : AppColors.controlBorder,
                width: 1,
            ),
        ),
        child: InkWell(
            borderRadius: BorderRadius.circular(6),
            onTap: () => _setMode(mode),
            child: Padding(
                padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
                child: Row(
                    children: [
                        Icon(icon,
                            size: 15,
                            color:
                                selected ? AppColors.primary : AppColors.textSecondary),
                        const SizedBox(width: 4),
                        Text(
                            label,
                            style: TextStyle(
                                fontSize: 13,
                                color: selected ? AppColors.primary : AppColors.textSecondary,
                                fontWeight: selected ? FontWeight.w600 : FontWeight.normal,

```

```

        ),
    ),
],
),
),
),
);
}
void _setMode(DisplayMode mode) {
    setState(() => _mode = mode);
    widget.onModeChanged?.call(mode.name);
    LogService.instance.info(
        LogTag.action,
        '切换歌词显示模式: ${mode.name}',
    );
}
Future<void> _switchVersion(Hymn hymn, String version) async {
    try {
        LogService.instance.info(
            LogTag.action,
            '切换音频版本: ${ChineseConvertService.instance.toSimplified(version)}',
            detail: '诗歌: 第 ${hymn.hymnNumber} 首《${hymn.title}》',
        );
        await widget.audio.switchAudioVersion(version);
        widget.onAudioVersionChanged?.call(version);
        if (mounted) setState(() {});
    } catch (e) {
        LogService.instance.error(
            LogTag.error,
            '切换音频版本异常',
            detail: '版本: $version\n 异常: $e',
        );
        _showAudioError();
    }
}
Widget _buildContent(Hymn? hymn) {
    if (hymn == null) {
        return Center(
            child: Text(
                '请在左侧选择一首诗歌',
                style: TextStyle(fontSize: 14, color: AppColors.textTertiary),
            ),
        );
    }
    switch (_mode) {
        case DisplayMode.lyrics:
            return _buildLyrics(hymn);
        case DisplayMode.numbered:
            return _buildScore(hymn.numberedPngPath, isEmpty: '暂无简谱');
        case DisplayMode.staff:

```

```

        return _buildScore(hymn.staffPngPath, isEmpty: '暂无五线谱');
    }
}
Widget _buildLyrics(Hymn hymn) {
    final verses = hymn.verses;
    if (verses.isEmpty) {
        return Center(
            child: Text(
                '暂无歌词',
                style: TextStyle(color: AppColors.textTertiary),
            ),
        );
    }
    return Container(
        color: AppColors.lyricsBg,
        child: LayoutBuilder(
            builder: (context, constraints) {
                const pad = 48.0;
                final verseTexts = verses.where((v) => v.trim().isNotEmpty).toList();
                int lineCount = 2;
                for (final v in verseTexts) {
                    final lines = v.split('\n').length;
                    lineCount += lines + 1;
                }
                final availH = (constraints.maxHeight - pad).clamp(100.0, 4000.0);
                final availW = constraints.maxWidth - pad;
                final maxByH = lineCount > 0 ? availH / (lineCount * 2.0) : 40.0;
                final maxByW = availW / 14.0;
                final ls = AppFonts.lyricsScale;
                final bodySize =
                    ([maxByH, maxByW].reduce((a, b) => a < b ? a : b) + 4.0)
                        .clamp(12.0, 100.0) *
                    ls;
                final titleSize = (bodySize * 1.4).clamp(20.0 * ls, 34.0 * ls);
                final labelSize = (bodySize * 0.7).clamp(12.0 * ls, 16.0 * ls);
                return SingleChildScrollView(
                    padding: const EdgeInsets.all(24),
                    child: ConstrainedBox(
                        constraints: BoxConstraints(
                            minHeight:
                                (constraints.maxHeight - pad).clamp(0.0, double.infinity),
                        ),
                        child: Column(
                            mainAxisAlignment: MainAxisAlignment.spaceBetween,
                            crossAxisAlignment: CrossAxisAlignment.center,
                            children: [
                                Text(
                                    ChineseConvertService.instance.toSimplified(hymn.title),
                                    style: TextStyle(
                                        fontSize: titleSize,

```

```

    }
  });
} else if (result == 'delete') {
  repo.deletePlaylist(pl.id);
  setState(() {
    if (_selectedPlaylist?.id == pl.id) {
      _selectedPlaylist = null;
      _selectedHymns = const [];
      _showContent = false;
    }
  });
}
}
}
}

class _EmptyHint extends StatelessWidget {
  final String text;
  const _EmptyHint({required this.text});
  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(Icons.music_off, size: 48, color: AppColors.textTertiary),
          const SizedBox(height: 12),
          Text(text,
            style: TextStyle(
              fontSize: 14, color: AppColors.textSecondary)),
        ],
      ),
    );
  }
}

import 'dart:io';
import 'package:path_provider/path_provider.dart';
import 'log_service.dart';
class AppPaths {
  static String? _dataRoot;
  static String? databasePath;
  static Future<void> init() async {
    if (_dataRoot != null) return;
    if (!isMobile) {
      final found = _locateDesktopDataRoot();
      if (found != null) {
        _dataRoot = found;
        databasePath = '$_dataRoot/tjc_hymn.db';
        LogService.instance
          .info(LogTag.lib, '数据根目录定位（桌面向上查找）', detail: databasePath);
      }
      return;
    }
  }
}

```

```

    }
    try {
        final dir = await getApplicationSupportDirectory();
        _dataRoot = dir.path;
        databasePath = '$_dataRoot/tjc_hymn.db';
        LogService.instance
            .info(LogTag.lib, '数据根目录定位（应用支持目录）', detail: databasePath);
    } catch (e) {
        _dataRoot = Directory.current.path;
        databasePath = '$_dataRoot/data/tjc_hymn.db';
        LogService.instance.warning(
            LogTag.lib,
            '数据根目录定位失败，回退当前工作目录',
            detail: '$databasePath\n 原因: $e',
        );
    }
}

static bool get isMobile =>
    !Platform.isWindows && !Platform.isLinux && !Platform.isMacOS;
static String? get dataRoot => _dataRoot;
static String get assetRoot => _dataRoot ?? '';
static String resolveAsset(String relative) {
    if (relative.isEmpty) return '';
    if (relative.contains(':\\')) return relative;
    if (relative.startsWith('file:')) return relative;
    String cleaned = relative.replaceFirst(RegExp(r'^\\.?/'), '');
    if (Platform.isWindows) {
        cleaned = cleaned.replaceAll('/', '\\');
    }
    return '$assetRoot${Platform.pathSeparator}$cleaned';
}

static String? _locateDesktopDataRoot() {
    var dir = Directory.current;
    final dirName = dir.path.split(Platform.pathSeparator).last;
    final isBundleDir = dirName.startsWith('echohymn_win_');
    for (var i = 0; i < 12; i++) {
        final candidate = '${dir.path}${Platform.pathSeparator}data';
        if (File('$candidate${Platform.pathSeparator}tjc_hymn.db').existsSync()) {
            return candidate;
        }
        if (isBundleDir) return null;
        final parent = dir.parent;
        if (parent.path == dir.path) break;
        dir = parent;
    }
    return null;
}

import 'dart:convert';
import 'dart:io';

```

```
class AppStateService {
    static const _fileName = 'state.json';
    static final AppStateService shared = AppStateService();
    Future<void> _writeChain = Future.value();
    static String get _statePath {
        try {
            final exe = Platform.resolvedExecutable;
            final dir = File(exe).parent.path;
            return '$dir\\$_fileName';
        } catch (_) {
            return _fileName;
        }
    }
}

Future<void> saveAll({
    required String leftTab,
    required String subcategory,
    required String playlistName,
    required String hymnNumber,
    required String audioVersion,
    required String displayMode,
    required int playlistIndex,
    bool showLeft = false,
    bool showRight = false,
    String appTheme = '',
    String fontSizeLevel = '',
    bool manualOnStart = true,
}) {
    final data = <String, Object>{
        'leftTab': leftTab,
        'subcategory': subcategory,
        'playlistName': playlistName,
        'hymnNumber': hymnNumber,
        'audioVersion': audioVersion,
        'displayMode': displayMode,
        'playlistIndex': playlistIndex,
        'showLeft': showLeft,
        'showRight': showRight,
        'appTheme': appTheme,
        'fontSizeLevel': fontSizeLevel,
        'manualOnStart': manualOnStart,
    };
    _writeChain = _writeChain.then((_) => _doWrite(data)).catchError((_) {});
    return _writeChain;
}

Future<void> updateManualOnStart(bool value) {
    _writeChain = _writeChain.then((_) async {
        final file = File(_statePath);
        Map<String, Object> data = {};
        if (await file.exists()) {
            try {
```

```

        final json = jsonDecode(await file.readAsString());
        if (json is Map<String, dynamic>) data = Map<String, Object>.from(json);
    } catch (_) {}
}
data['manualOnStart'] = value;
const encoder = JsonEncoder.withIndent(' ');
final tmp = File('${_statePath}.tmp');
await tmp.writeAsString(encoder.convert(data), flush: true);
try {
    await tmp.rename(file.path);
} catch (_) {
    if (await file.exists()) await file.delete();
    await tmp.rename(file.path);
}
}).catchError((_) {});
return _writeChain;
}
Future<void> _doWrite(Map<String, Object> data) async {
    final file = File(_statePath);
    final tmp = File('${_statePath}.tmp');
    const encoder = JsonEncoder.withIndent(' ');
    await tmp.writeAsString(encoder.convert(data), flush: true);
    try {
        await tmp.rename(file.path);
    } catch (_) {
        if (await file.exists()) {
            await file.delete();
        }
        await tmp.rename(file.path);
    }
}
Future<AppState> load() async {
    try {
        final file = File(_statePath);
        if (!await file.exists()) {
            return _defaultState;
        }
        final text = await file.readAsString();
        final json = jsonDecode(text);
        if (json is! Map<String, dynamic>) return _defaultState;
        return AppState(
            leftTab: (json['leftTab'] as String?) ?? '',
            subcategory: (json['subcategory'] as String?) ?? '',
            playlistName: (json['playlistName'] as String?) ?? '',
            hymnNumber: (json['hymnNumber'] as String?) ?? '',
            audioVersion: (json['audioVersion'] as String?) ?? '',
            displayMode: (json['displayMode'] as String?) ?? '',
            playlistIndex: (json['playlistIndex'] as num?)?.toInt() ?? -1,
            showLeft: (json['showLeft'] as bool?) ?? false,
            showRight: (json['showRight'] as bool?) ?? false,

```

```

        appTheme: (json['appTheme'] as String?) ?? '',
        fontSizeLevel: (json['fontSizeLevel'] as String?) ?? '',
        manualOnStart: (json['manualOnStart'] as bool?) ?? true,
    );
} catch (_) {
    return _defaultState;
}
}
static const _defaultState = AppState(
    leftTab: '',
    subcategory: '',
    playlistName: '',
    hymnNumber: '',
    audioVersion: '',
    displayMode: '',
    playlistIndex: -1,
);
}
class AppState {
    final String leftTab;
    final String subcategory;
    final String playlistName;
    final String hymnNumber;
    final String audioVersion;
    final String displayMode;
    final int playlistIndex;
    final bool showLeft;
    final bool showRight;
    final String appTheme;
    final String fontSizeLevel;
    final bool manualOnStart;
    const AppState({
        required this.leftTab,
        required this.subcategory,
        required this.playlistName,
        required this.hymnNumber,
        required this.audioVersion,
        required this.displayMode,
        required this.playlistIndex,
        this.showLeft = false,
        this.showRight = false,
        this.appTheme = '',
        this.fontSizeLevel = '',
        this.manualOnStart = true,
    });
}
import 'dart:async';
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';

```

```

import '../models/hymn.dart';
import 'app_paths.dart';
import 'log_service.dart';
const MethodChannel _windowChannel = MethodChannel('echo_hymn/window');
enum PlayerStatus { idle, loading, playing, paused, error }
class AudioService {
  static AudioService? instance;
  final AudioPlayer _player = AudioPlayer();
  final StreamController<PlayerStatus> _statusCtrl =
    StreamController<PlayerStatus>.broadcast();
  final StreamController<Duration> _positionCtrl =
    StreamController<Duration>.broadcast();
  final StreamController<Duration> _durationCtrl =
    StreamController<Duration>.broadcast();
  Hymn? _currentHymn;
  String _currentAudioVersion = '鋼琴版';
  String _actualAudioVersion = '鋼琴版';
  bool _disposed = false;
  String? lastError;
  double _volume = 1.0;
  bool _muted = false;
  final ValueNotifier<double> volumeNotifier = ValueNotifier<double>(1.0);
  final ValueNotifier<bool> mutedNotifier = ValueNotifier<bool>(false);
  DateTime _lastSwitchAt = DateTime.fromMillisecondsSinceEpoch(0);
  void Function()? onCurrentChanged;
  List<Hymn> _playlist = const [];
  int _currentIndex = -1;
  AudioService() {
    instance = this;
    LogService.instance
      .info(LogTag.lib, '音频播放库加载完成 (audioplayers / Windows Media Foundation) ');
    _init();
  }
  Future<void> _init() async {
    _player.onDurationChanged.listen((d) {
      if (!_durationCtrl.isClosed) _durationCtrl.add(d);
    });
    _player.onPositionChanged.listen((p) {
      if (!_positionCtrl.isClosed) _positionCtrl.add(p);
    });
    _player.onPlayerStateChanged.listen((state) {
      switch (state) {
        case PlayerState.playing:
          _emitStatus(PlayerStatus.playing);
          break;
        case PlayerState.paused:
        case PlayerState.stopped:
        case PlayerState.completed:
          _emitStatus(PlayerStatus.paused);
          break;
      }
    });
  }
  void _emitStatus(PlayerStatus status) {
    _statusCtrl.add(status);
  }
}

```

```

        case PlayerState.disposed:
            break;
    }
});
_player.onPlayerComplete.listen((_) {
    playNext();
});
}
Stream<PlayerStatus> get statusStream => _statusCtrl.stream;
Stream<Duration> get positionStream => _positionCtrl.stream;
Stream<Duration> get durationStream => _durationCtrl.stream;
Hymn? get currentHymn => _currentHymn;
String get currentAudioVersion => _actualAudioVersion;
String get preferredAudioVersion => _currentAudioVersion;
int get currentIndex => _currentIndex;
bool get isPlaying => _player.state == PlayerState.playing;
List<Hymn> get playlist => _playlist;
List<String> availableVersions(Hymn hymn) {
    if (hymn.audioVersionList.isNotEmpty) return hymn.audioVersionList;
    if (hymn.audioVersions.isNotEmpty) return hymn.audioVersions.keys.toList();
    return const [];
}
void setPlaylist(List<Hymn> hymns, {int? startIndex}) {
    _playlist = hymns;
    if (startIndex != null && startIndex >= 0 && startIndex < hymns.length) {
        _currentIndex = startIndex;
    }
}
Future<void> loadHymn(Hymn hymn, {int? index, String? version}) async {
    _currentHymn = hymn;
    if (index != null) _currentIndex = index;
    if (version != null) {
        _currentAudioVersion = version;
        _actualAudioVersion = version;
    }
    onCurrentChanged?.call();
    await _player.stop();
    _emitStatus(PlayerStatus.idle);
    LogService.instance.info(
        LogTag.play,
        '加载诗歌（不播放）：第 ${hymn.hymnNumber} 首《${hymn.title}》',
        detail: '音频版本: $version / 列表位置: $index',
    );
}
Future<void> playHymn(Hymn hymn, {int? index, String? version}) async {
    _currentHymn = hymn;
    if (index != null) _currentIndex = index;
    onCurrentChanged?.call();
    var audioVersion = version ?? _currentAudioVersion;
    if (hymn.audioVersions.containsKey(audioVersion)) {

```

```
        _currentAudioVersion = audioVersion;
        _actualAudioVersion = audioVersion;
    } else if (hymn.audioVersionList.isNotEmpty) {
        audioVersion = hymn.audioVersionList.first;
        _actualAudioVersion = audioVersion;
    }
    final rel = hymn.audioVersions[audioVersion];
    if (rel == null || rel.isEmpty) {
        lastError = '无音频文件';
        _emitStatus(PlayerStatus.error);
        LogService.instance.error(
            LogTag.play,
            '播放失败: 第 ${hymn.hymnNumber} 首《${hymn.title}》无音频文件',
            detail: '当前音频版本: $audioVersion / 可用版本: ${hymn.audioVersionList}',
        );
        return;
    }
    final abs = AppPaths.resolveAsset(rel);
    if (abs.isEmpty) {
        lastError = '音频路径为空';
        _emitStatus(PlayerStatus.error);
        LogService.instance.error(
            LogTag.play,
            '播放失败: 第 ${hymn.hymnNumber} 首《${hymn.title}》音频路径为空',
            detail: '音频版本: $audioVersion / 相对路径: $rel',
        );
        return;
    }
    LogService.instance.info(
        LogTag.play,
        '播放诗歌: 第 ${hymn.hymnNumber} 首《${hymn.title}》',
        detail: '音频版本: $audioVersion\n 文件路径: $abs',
    );
    _emitStatus(PlayerStatus.loading);
    try {
        await _player.stop();
        if (abs.startsWith('http://') || abs.startsWith('https://')) {
            await _player.play(UrlSource(abs));
        } else {
            await _player.play(DeviceFileSource(abs));
        }
    }
    lastError = null;
    _emitStatus(PlayerStatus.playing);
    LogService.instance.info(
        LogTag.play,
        '播放成功: 第 ${hymn.hymnNumber} 首《${hymn.title}》',
        detail: '音频版本: $audioVersion',
    );
} catch (e) {
    lastError = e.toString();
}
```

```

        _emitStatus(PlayerStatus.error);
        LogService.instance.error(
            LogTag.play,
            '播放异常: 第 ${hymn.hymnNumber} 首 《${hymn.title}》 ',
            detail: '音频版本: $audioVersion\n 文件路径: $abs\n 异常: $e',
        );
    }
}

Future<void> switchAudioVersion(String version) async {
    final h = _currentHymn;
    if (h == null) return;
    await playHymn(h, index: _currentIndex, version: version);
}

Future<void> playAt(int index) async {
    final now = DateTime.now();
    if (now.difference(_lastSwitchAt) < const Duration(milliseconds: 250)) {
        return;
    }
    _lastSwitchAt = now;
    if (index < 0 || index >= _playlist.length) return;
    await playHymn(_playlist[index], index: index);
}

Future<void> togglePlayPause() async {
    if (_currentHymn == null) {
        if (_playlist.isNotEmpty) {
            await playAt(0);
        }
        return;
    }
    final h = _currentHymn!;
    if (_player.state == PlayerState.playing) {
        await _player.pause();
        LogService.instance
            .info(LogTag.play, '暂停播放: 第 ${h.hymnNumber} 首 《${h.title}》 ');
    } else {
        if (_player.state == PlayerState.stopped ||
            _player.state == PlayerState.completed) {
            LogService.instance
                .info(LogTag.play, '重新播放: 第 ${h.hymnNumber} 首 《${h.title}》 ');
            await playHymn(h, index: _currentIndex);
        } else {
            await _player.resume();
            LogService.instance
                .info(LogTag.play, '继续播放: 第 ${h.hymnNumber} 首 《${h.title}》 ');
        }
    }
}

Future<void> playNext() async {
    if (_playlist.isEmpty) return;
    final next = _currentIndex < 0 ? 0 : (_currentIndex + 1) % _playlist.length;

```

```

    LogService.instance
        .info(LogTag.play, '切换下一首 → 列表位置 $next/${_playlist.length - 1}');
    await playAt(next);
}
Future<void> playPrev() async {
    if (_playlist.isEmpty) return;
    final prev = _currentIndex < 0
        ? 0
        : (_currentIndex - 1 + _playlist.length) % _playlist.length;
    LogService.instance
        .info(LogTag.play, '切换上一首 → 列表位置 $prev/${_playlist.length - 1}');
    await playAt(prev);
}
Future<void> seek(Duration position) async {
    await _player.seek(position);
}
double get volume => _volume;
bool get muted => _muted;
bool _systemVolumeAvailable = false;
double get _gain =>
    _muted ? 0 : (_systemVolumeAvailable ? 1.0 : _volume);
Future<void> setVolume(double volume) async {
    final v = volume.clamp(0.0, 1.0);
    if (v > 0 && _muted) {
        _muted = false;
        mutedNotifier.value = false;
    }
    _volume = v;
    volumeNotifier.value = v;
    await _player.setVolume(_gain);
    LogService.instance
        .info(LogTag.play, '音量设置为 ${(_volume * 100).round()}%');
    _syncToSystem();
}
Future<void> changeVolume(double delta) => setVolume(_volume + delta);
Future<void> toggleMute() async {
    _muted = !_muted;
    mutedNotifier.value = _muted;
    await _player.setVolume(_gain);
    LogService.instance.info(LogTag.play,
        _muted ? '静音开启' : '静音取消 (音量 ${(_volume * 100).round()}%)');
    _syncToSystem();
}
Future<void> loadSystemVolume() async {
    try {
        final result = await _windowChannel
            .invokeMethod<Map<dynamic, dynamic>>('getSystemVolume');
        if (result == null) return;
        final v = (result['volume'] as num?)?.toDouble();
        final m = result['muted'] as bool? ?? false;

```

```

        if (v == null) return;
        _volume = v.clamp(0.0, 1.0);
        volumeNotifier.value = _volume;
        _muted = m;
        mutedNotifier.value = _muted;
        _systemVolumeAvailable = true;
        await _player.setVolume(_gain);
        LogService.instance.info(LogTag.play,
            '初始化为系统音量 ${(_volume * 100).round()}%'
            '${_muted ? '（系统静音）' : ''}');
        _startSystemVolumePoll();
    } catch (_) {
    }
}

Timer? _sysVolPollTimer;
void _startSystemVolumePoll() {
    _sysVolPollTimer?.cancel();
    _sysVolPollTimer =
        Timer.periodic(const Duration(seconds: 1), (_) => _pollSystemVolume());
}

Future<void> _pollSystemVolume() async {
    try {
        final result = await _windowChannel
            .invokeMethod<Map<dynamic, dynamic>>('getSystemVolume');
        if (result == null) return;
        final v = (result['volume'] as num?).toDouble();
        final m = result['muted'] as bool? ?? false;
        if (v == null) return;
        if ((v - _volume).abs() > 0.02 || m != _muted) {
            _volume = v.clamp(0.0, 1.0);
            volumeNotifier.value = _volume;
            _muted = m;
            mutedNotifier.value = _muted;
            await _player.setVolume(_gain);
            LogService.instance.info(LogTag.play,
                '跟随系统音量 ${(_volume * 100).round()}%'
                '${_muted ? '（静音）' : ''}');
        }
    } catch (_) {
    }
}

void _syncToSystem() {
    unawaited(_windowChannel
        .invokeMethod<void>('setSystemVolume', <String, dynamic>{
            'volume': _volume,
            'muted': _muted,
        }).catchError((_) {}));
}

Future<void> stop() async {
    await _player.stop();
}

```

```

    _emitStatus(PlayerStatus.idle);
    final h = _currentHymn;
    if (h != null) {
        LogService.instance
            .info(LogTag.play, '停止播放: 第 ${h.hymnNumber} 首《${h.title}》');
    }
}

void _emitStatus(PlayerStatus status) {
    if (!_statusCtrl.isClosed) _statusCtrl.add(status);
}

Future<void> dispose() async {
    if (_disposed) return;
    _disposed = true;
    if (instance == this) instance = null;
    _sysVolPollTimer?.cancel();
    volumeNotifier.dispose();
    mutedNotifier.dispose();
    await _player.dispose();
    await _statusCtrl.close();
    await _positionCtrl.close();
    await _durationCtrl.close();
}

}

String formatTime(Duration d) {
    final m = d.inMinutes;
    final s = d.inSeconds % 60;
    return '$m:${s.toString().padLeft(2, '0')}';
}

import '../data/chinese_convert_map.dart';
String toSimplifiedImpl(String text) {
    if (text.isEmpty) return text;
    final buf = StringBuffer();
    for (final rune in text.runes) {
        final ch = String.fromCharCode(rune);
        buf.write(kToSimplifiedByChar[ch] ?? ch);
    }
    return buf.toString();
}

String toTraditionalImpl(String text) {
    if (text.isEmpty) return text;
    final buf = StringBuffer();
    for (final rune in text.runes) {
        final ch = String.fromCharCode(rune);
        buf.write(kToTraditionalByChar[ch] ?? ch);
    }
    return buf.toString();
}

void initOpencclImpl(String dir) {}
import '../data/chinese_convert_map.dart';
import 'chinese_convert_stub.dart'

```

```

    if (dart.library.io) 'chinese_convert_native.dart' as impl;
import 'log_service.dart';
class ChineseConvertService {
  ChineseConvertService._() {
    LogService.instance.info(
      LogTag.lib,
      '简繁转换映射表加载完成（纯 Dart 查表）',
      detail: '繁→简 ${kToSimplifiedByChar.length} 字 / 简→繁 '
        '${kToTraditionalByChar.length} 字',
    );
  }
  static final ChineseConvertService instance = ChineseConvertService._();
  void initOpenccc(String dictDir) => impl.initOpencccImpl(dictDir);
  String toSimplified(String text) => impl.toSimplifiedImpl(text);
  String toTraditional(String text) => impl.toTraditionalImpl(text);
}
String toSimplifiedImpl(String text) => text;
String toTraditionalImpl(String text) => text;
void initOpencccImpl(String dir) {}
import '../models/hymn.dart';
import 'chinese_convert_service.dart';
class HymnSearchHit {
  final Hymn hymn;
  final bool titleMatched;
  final bool verseMatched;
  final String displayVerse;
  const HymnSearchHit({
    required this.hymn,
    required this.titleMatched,
    required this.verseMatched,
    required this.displayVerse,
  });
}
class HymnSearchService {
  HymnSearchService._();
  static String normalizedKeyword(String keyword) =>
    ChineseConvertService.instance.toSimplified(keyword.trim());
  static bool _matches(String rawText, String kwS, String kwT) {
    if (kwS.isNotEmpty &&
      ChineseConvertService.instance.toSimplified(rawText).contains(kwS)) {
      return true;
    }
    return kwT.isNotEmpty && kwT != kwS && rawText.contains(kwT);
  }
  static List<HymnSearchHit> search(List<Hymn> all, String keyword) {
    final kw = keyword.trim();
    if (kw.isEmpty) return const [];
    final kwS = ChineseConvertService.instance.toSimplified(kw);
    final kwT = ChineseConvertService.instance.toTraditional(kw);
    final titleHits = <HymnSearchHit>[];

```

```

final verseHits = <HymnSearchHit>[];
for (final h in all) {
  final titleMatched = _matches(h.title, kwS, kwT);
  var matchedVerse = -1;
  for (var i = 0; i < h.verses.length; i++) {
    if (_matches(h.verses[i], kwS, kwT)) {
      matchedVerse = i;
      break;
    }
  }
  final verseMatched = matchedVerse >= 0;
  if (!titleMatched && !verseMatched) continue;
  final rawVerse = verseMatched
    ? h.verses[matchedVerse]
    : (h.verses.isNotEmpty ? h.verses.first : '');
  var display =
    rawVerse.trim().isEmpty ? '' : _toSimplifiedMultiline(rawVerse);
  if (verseMatched && display.isNotEmpty) {
    display = _windowAroundKeyword(display, kwS);
  }
  final hit = HymnSearchHit(
    hymn: h,
    titleMatched: titleMatched,
    verseMatched: verseMatched,
    displayVerse: display,
  );
  (titleMatched ? titleHits : verseHits).add(hit);
}
return [...titleHits, ...verseHits];
}

static String _windowAroundKeyword(String simplified, String kwS) {
  if (kwS.isEmpty) return simplified;
  var i = simplified.indexOf(kwS);
  if (i < 0) return simplified;
  if (i == 0) return simplified;
  final lineStart = simplified.lastIndexOf('\n', i - 1) + 1;
  if (lineStart == 0) return simplified;
  return '...${simplified.substring(lineStart)}';
}

static String _toSimplifiedMultiline(String text) =>
  ChineseConvertService.instance.toSimplified(text);
}

import 'dart:io';
enum LogLevel { info, warning, error }
abstract final class LogTag {
  static const String lib = 'LIB';
  static const String ui = 'UI';
  static const String action = 'ACTION';
  static const String playlist = 'PLAYLIST';
  static const String play = 'PLAY';

```

```

    static const String error = 'ERROR';
    static const String system = 'SYS';
}
class LogService {
    static final LogService instance = LogService._();
    LogService._();
    Future<void> _writeChain = Future.value();
    bool _initialized = false;
    String? _logDirPath;
    String? _currentFileName;
    Future<void> init({bool ensureExeDir = true}) async {
        if (_initialized) return;
        _initialized = true;
        try {
            String base;
            if (ensureExeDir) {
                try {
                    final exe = Platform.resolvedExecutable;
                    base = File(exe).parent.path;
                } catch (_) {
                    base = Directory.current.path;
                }
            } else {
                base = Directory.current.path;
            }
            final dir = Directory('$base${Platform.pathSeparator}logs');
            await dir.create(recursive: true);
            _logDirPath = dir.path;
        } catch (e) {
            _logDirPath = null;
        }
    }
    String? get logDirPath => _logDirPath;
    void log(LogLevel level, String tag, String message, {String? detail}) {
        final line = _format(level, tag, message, detail);
        _writeChain = _writeChain.then((_) => _writeln(line)).catchError((_) {});
    }
    void info(String tag, String message, {String? detail}) =>
        log(LogLevel.info, tag, message, detail: detail);
    void warning(String tag, String message, {String? detail}) =>
        log(LogLevel.warning, tag, message, detail: detail);
    void error(String tag, String message, {String? detail}) =>
        log(LogLevel.error, tag, message, detail: detail);
    void exception(String message, Object e, [StackTrace? stack]) {
        final detail = StringBuffer()
            ..writeln('异常类型: ${e.runtimeType}')
            ..writeln('异常信息: $e');
        if (stack != null) {
            detail.writeln('--- 堆栈 ---');
            detail.write(stack.toString());
        }
    }
}

```

```

    }
    log(LogLevel.error, LogTag.error, message, detail: detail.toString());
}
String _format(LogLevel level, String tag, String message, String? detail) {
    final now = DateTime.now();
    String two(int v) => v.toString().padLeft(2, '0');
    final ts = '${now.year}-${two(now.month)}-${two(now.day)} '
        '${two(now.hour)}:${two(now.minute)}:${two(now.second)}.'
        '${now.millisecond.toString().padLeft(3, '0')}';
    final lv = switch (level) {
        LogLevel.info => 'INFO ',
        LogLevel.warning => 'WARN ',
        LogLevel.error => 'ERROR',
    };
    final buf = StringBuffer('[${ts}] [${lv}] [${tag}] $message');
    if (detail != null && detail.trim().isNotEmpty) {
        final indented = detail.split('\n').map((l) => '    | $l').join('\n');
        buf.write('\n$indented');
    }
    return buf.toString();
}
Future<void> _writeLine(String line) async {
    final dir = _logDirPath;
    if (dir == null) return;
    final fileName = _fileNameFor(DateTime.now());
    if (_currentFileName == null || _currentFileName != fileName) {
        _currentFileName = fileName;
        await _rotateAndCleanup(dir, fileName);
    }
    final file = File('${dir}${Platform.pathSeparator}$fileName');
    try {
        if (!await file.exists()) {
            await file.writeAsString('\uFEFF', flush: true);
        }
        await file.writeAsString('$line${Platform.lineTerminator}',
            mode: FileMode.append, flush: true);
    } catch (_) {}
}
String _fileNameFor(DateTime d) {
    String two(int v) => v.toString().padLeft(2, '0');
    return 'app_${d.year}-${two(d.month)}-${two(d.day)}.log';
}
Future<void> _rotateAndCleanup(String dir, String currentName) async {
    final dirObj = Directory(dir);
    try {
        final files = dirObj
            .listSync()
            .whereType<File>()
            .where((f) =>

```

```

        f.path.split(Platform.pathSeparator).last.startsWith('app_'))
      .toList();
    final sorted = files.toList()..sort((a, b) => b.path.compareTo(a.path));
    const maxFiles = 7;
    if (sorted.length > maxFiles) {
      for (final f in sorted.skip(maxFiles)) {
        try {
          await f.delete();
        } catch (_) {}
      }
    }
  } catch (_) {}
}

import 'dart:convert';
import 'package:sqlite3/sqlite3.dart';
import '../models/hymn.dart';
import '../models/hymn_category.dart';
import '../models/playlist.dart';
import 'app_paths.dart';
import 'log_service.dart';
class SqliteRepository {
  final Database _db;
  final bool hasCoreTables;
  SqliteRepository._(this._db) : hasCoreTables = _probeCoreTables(_db);
  static bool _probeCoreTables(Database db) {
    try {
      final rows = db.select(
        "SELECT name FROM sqlite_master WHERE type='table' AND name IN ('tjc_hymn','hymn_category')");
      return rows.length == 2;
    } catch (_) {}
    return false;
  }
}

static Future<SqliteRepository> open() async {
  await AppPaths.init();
  final path = AppPaths.databasePath ?? '';
  final db = sqlite3.open(path);
  LogService.instance.info(LogTag.lib, '打开 SQLite 数据库', detail: path);
  final repo = SqliteRepository._(db);
  repo._ensurePlaylistTable();
  try {
    final hymnCount =
      db.select('SELECT COUNT(*) AS c FROM tjc_hymn').first['c'];
    final catCount =
      db.select('SELECT COUNT(*) AS c FROM hymn_category').first['c'];
    final plCount =
      db.select('SELECT COUNT(*) AS c FROM playlist_hymn').first['c'];
  }
}

```

```

        LogService.instance.info(
            LogTag.lib,
            '数据库数据加载完成',
            detail: '诗歌 $hymnCount 首 / 分类 $catCount 条 / 个人歌单 $plCount 个',
        );
    } catch (e) {
        LogService.instance.warning(LogTag.lib, '数据库统计失败', detail: '$e');
    }
    return repo;
}

void _ensurePlaylistTable() {
    const newTableSql = '''
        CREATE TABLE IF NOT EXISTS playlist_hymn (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            hymns TEXT NOT NULL DEFAULT '[]',
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL
        )
    ''';
    final hasOldPlaylist = _tableExists('playlist');
    final hasPlaylistHymn = _tableExists('playlist_hymn');
    final currentIsNew =
        hasPlaylistHymn && _tableHasColumn('playlist_hymn', 'hymns');
    if (hasOldPlaylist && hasPlaylistHymn && !currentIsNew) {
        _db.execute('DROP TABLE IF EXISTS playlist_hymn_old');
        _db.execute('ALTER TABLE playlist_hymn RENAME TO playlist_hymn_old');
        _db.execute(newTableSql);
        _migrateFromOldTwoTables();
    } else if (hasOldPlaylist && !hasPlaylistHymn) {
        _db.execute(newTableSql);
        _migrateFromOldTwoTables();
    } else if (!hasPlaylistHymn) {
        _db.execute(newTableSql);
    } else if (hasOldPlaylist && currentIsNew) {
        _db.execute('DROP TABLE IF EXISTS playlist');
        _db.execute('DROP TABLE IF EXISTS playlist_hymn_old');
    }
}

bool _tableExists(String name) {
    final rows = _db.select(
        "SELECT name FROM sqlite_master WHERE type='table' AND name = ?",
        [name]);
    return rows.isNotEmpty;
}

bool _tableHasColumn(String table, String column) {
    final info = _db.select('PRAGMA table_info($table)');
    return info.any((r) => (r['name'] as String?) == column);
}

void _migrateFromOldTwoTables() {

```

```

final oldRows = _db
    .select('SELECT * FROM playlist ORDER BY id')
    .toList(growable: false);
for (final row in oldRows) {
    final id = (row['id'] as int?) ?? 0;
    final name = (row['name'] as String?) ?? '';
    final createdAt = (row['created_at'] as String?) ?? '';
    final items = _db.select(
        'SELECT * FROM playlist_hymn_old WHERE playlist_id = ? ORDER BY sort_order',
        [id],
    ).toList(growable: false);
    final hymnList = <Map<String, Object>>[];
    for (final it in items) {
        final hymnId = (it['hymn_id'] as int?) ?? 0;
        final itemName = (it['name'] as String?) ?? '';
        if (hymnId > 0) {
            hymnList.add({itemName: hymnId});
        }
    }
    _db.execute(
        'INSERT OR REPLACE INTO playlist_hymn (id, name, hymns, created_at, updated_at) VALUES (
?, ?, ?, ?, ?)',
        [id, name, jsonEncode(hymnList), createdAt, createdAt],
    );
}
_db.execute('DROP TABLE IF EXISTS playlist');
_db.execute('DROP TABLE IF EXISTS playlist_hymn_old');
}
List<Hymn> getAllHymns() {
    if (!hasCoreTables) return const [];
    final rows = _db.select(
        'SELECT * FROM tjc_hymn ORDER BY CAST(hymn_number AS INTEGER), hymn_number');
    return rows.map((r) => Hymn.fromDbRow(_rowToMap(r))).toList();
}
List<Hymn> searchHymns(String keyword) {
    if (!hasCoreTables) return const [];
    final kw = keyword.trim();
    if (kw.isEmpty) return getAllHymns();
    final like = '%$kw%';
    final isNum = int.tryParse(kw) != null;
    if (isNum) {
        final rows = _db.select(
            'SELECT * FROM tjc_hymn WHERE hymn_number = ? OR hymn_number LIKE ? ORDER BY CAST(hymn_n
umber AS INTEGER)',
            [kw, '$kw%'],
        );
        return rows.map((r) => Hymn.fromDbRow(_rowToMap(r))).toList();
    }
    final rows = _db.select(
        'SELECT * FROM tjc_hymn WHERE title LIKE ? ORDER BY CAST(hymn_number AS INTEGER)',

```

```

        [like],
    );
    return rows.map((r) => Hymn.fromDbRow(_rowToMap(r))).toList();
}

List<Hymn> getHymnsPage(int offset, int limit) {
    if (!hasCoreTables) return const [];
    final rows = _db.select(
        'SELECT * FROM tjc_hymn ORDER BY CAST(hymn_number AS INTEGER) LIMIT ? OFFSET ?',
        [limit, offset],
    );
    return rows.map((r) => Hymn.fromDbRow(_rowToMap(r))).toList();
}

Hymn? hymnByNumber(String number) {
    if (!hasCoreTables) return null;
    final rows = _db.select(
        'SELECT * FROM tjc_hymn WHERE hymn_number = ? LIMIT 1',
        [number],
    );
    if (rows.isEmpty) return null;
    return Hymn.fromDbRow(_rowToMap(rows.first));
}

Hymn? hymnById(int id) {
    if (!hasCoreTables) return null;
    final rows = _db.select(
        'SELECT * FROM tjc_hymn WHERE id = ? LIMIT 1',
        [id],
    );
    if (rows.isEmpty) return null;
    return Hymn.fromDbRow(_rowToMap(rows.first));
}

List<HymnCategory> getAllCategories() {
    if (!hasCoreTables) return const [];
    final rows = _db.select('SELECT * FROM hymn_category ORDER BY id');
    return rows.map((r) => HymnCategory.fromDbRow(_rowToMap(r))).toList();
}

Map<String, List<HymnCategory>> buildCategoryTree() {
    final map = <String, List<HymnCategory>>{};
    for (final c in getAllCategories()) {
        map.putIfAbsent(c.category, () => []).add(c);
    }
    return map;
}

List<Playlist> getPlaylists() {
    final rows = _db.select('SELECT * FROM playlist_hymn ORDER BY id');
    return rows.map((r) => Playlist.fromDbRow(_rowToMap(r))).toList();
}

Playlist? getPlaylistById(int id) {
    final rows =
        _db.select('SELECT * FROM playlist_hymn WHERE id = ? LIMIT 1', [id]);
    if (rows.isEmpty) return null;

```

```

        return Playlist.fromDbRow(_rowToMap(rows.first));
    }
    int createPlaylist(String name, [List<MapEntry<String, int>> hymns = const []]) {
        final now = DateTime.now().toIso8601String();
        final json = Playlist.hymnsToJson(hymns);
        _db.execute(
            'INSERT INTO playlist_hymn (name, hymns, created_at, updated_at) VALUES (?, ?, ?, ?)',
            [name, json, now, now],
        );
        final id = _db.lastInsertRowId;
        LogService.instance.info(
            LogTag.playlist,
            '创建个人歌单',
            detail: '歌单 ID: $id\n 歌单名称: $name\n 诗歌数量: ${hymns.length}\n 成员明细: $json\n 创建时间: $now',
        );
        return id;
    }
    void updatePlaylist(
        int id,
        String name,
        List<MapEntry<String, int>> hymns,
    ) {
        final now = DateTime.now().toIso8601String();
        final json = Playlist.hymnsToJson(hymns);
        _db.execute(
            'UPDATE playlist_hymn SET name = ?, hymns = ?, updated_at = ? WHERE id = ?',
            [name, json, now, id],
        );
        LogService.instance.info(
            LogTag.playlist,
            '修改个人歌单',
            detail: '歌单 ID: $id\n 歌单名称: $name\n 诗歌数量: ${hymns.length}\n'
                '成员明细: $json\n 更新时间: $now',
        );
    }
    void renamePlaylist(int id, String newName) {
        final now = DateTime.now().toIso8601String();
        _db.execute(
            'UPDATE playlist_hymn SET name = ?, updated_at = ? WHERE id = ?',
            [newName, now, id],
        );
        LogService.instance.info(
            LogTag.playlist,
            '重命名个人歌单',
            detail: '歌单 ID: $id\n 新名称: $newName\n 更新时间: $now',
        );
    }
    void deletePlaylist(int id) {
        final before = getPlaylistById(id);

```

```

        _db.execute('DELETE FROM playlist_hymn WHERE id = ?', [id]);
        LogService.instance.info(
            LogTag.playlist,
            '删除个人歌单',
            detail: before == null
                ? '歌单 ID: $id (删除前未能读取到歌单信息)'
                : '歌单 ID: $id\n 歌单名称: ${before.name}\n'
                    '诗歌数量: ${before.hymns.length}\n 成员明细: ${Playlist.hymnsToJson(before.hymns)}'
        ),
    );
}
Map<String, Object?> _rowToMap(Row row) => Map<String, Object?>.from(row);
void dispose() {
    _db.dispose();
}
}
import 'package:flutter/material.dart';
import '../services/log_service.dart';
enum FontSizeLevel {
    normal('normal', '默认', 1.0),
    medium('medium', '中号', 1.3),
    large('large', '大号', 1.6),
    xlarge('xlarge', '最大', 1.9);
    const FontSizeLevel(this.id, this.label, this.scale);
    final String id;
    final String label;
    final double scale;
}
class FontScaleController {
    FontScaleController._();
    static final FontScaleController instance = FontScaleController._();
    final ValueNotifier<FontSizeLevel> notifier =
        ValueNotifier(FontSizeLevel.normal);
    FontSizeLevel get current => notifier.value;
    double get scale => current.scale;
    void switchTo(FontSizeLevel level) {
        if (level == current) return;
        LogService.instance
            .info(LogTag.action, '切换字号等级: ${level.label} (×${level.scale})');
        notifier.value = level;
    }
}
class AppFonts {
    static double get scale => FontScaleController.instance.scale;
    static double get lyricsScale => scale;
}
FontSizeLevel fontSizeLevelById(String? id) {
    return FontSizeLevel.values.firstWhere(
        (l) => l.id == id,
        orElse: () => FontSizeLevel.normal,
    );
}

```

```
);  
}  
import 'package:flutter/material.dart';  
class AppPalette {  
  final String id;  
  final String name;  
  final Brightness brightness;  
  final Color primary;  
  final Color primaryHover;  
  final Color accent;  
  final Color pageBg;  
  final Color cardBg;  
  final Color titleBarBg;  
  final Color topBarBg;  
  final Color sidebarBg;  
  final Color rightPanelBg;  
  final Color versionBarBg;  
  final Color playBarBg;  
  final Color statusBarBg;  
  final Color controlBg;  
  final Color controlBorder;  
  final Color lyricsBg;  
  final Color textPrimary;  
  final Color textSecondary;  
  final Color textTertiary;  
  final Color divider;  
  final Color border;  
  final Color success;  
  final Color warning;  
  final Color danger;  
  final Color selectedBg;  
  final Color scrollbarThumb;  
  final Color windowBtnHover;  
  const AppPalette({  
    required this.id,  
    required this.name,  
    this.brightness = Brightness.light,  
    required this.primary,  
    required this.primaryHover,  
    required this.accent,  
    required this.pageBg,  
    required this.cardBg,  
    required this.titleBarBg,  
    required this.topBarBg,  
    required this.sidebarBg,  
    required this.rightPanelBg,  
    required this.versionBarBg,  
    required this.playBarBg,  
    required this.statusBarBg,  
    required this.controlBg,
```

```
        required this.controlBorder,
        required this.lyricsBg,
        required this.textPrimary,
        required this.textSecondary,
        required this.textTertiary,
        required this.divider,
        required this.border,
        required this.success,
        required this.warning,
        required this.danger,
        required this.selectedBg,
        required this.scrollbarThumb,
        required this.windowBtnHover,
    });
}

const kPaletteMorning = AppPalette(
    id: 'morningBlue',
    name: '晨光蓝 · 经典',
    primary: Color(0xFF2B6AE0),
    primaryHover: Color(0xFF1E57C8),
    accent: Color(0xFF00A870),
    pageBg: Color(0xFFFAFBFC),
    cardBg: Color(0xFFFFFFFF),
    titleBarBg: Color(0xFFFFFFFF),
    topBarBg: Color(0xFFF5F8FF),
    sidebarBg: Color(0xFFEBF1FA),
    rightPanelBg: Color(0xFFF1F4FA),
    versionBarBg: Color(0xFFE4EFFC),
    playBarBg: Color(0xFFFBFCFE),
    statusBarBg: Color(0xFFF2F5FA),
    controlBg: Color(0xFFEDF3FB),
    controlBorder: Color(0xFF98A1B2),
    lyricsBg: Color(0xFFF2F6FD),
    textPrimary: Color(0xFF1F2329),
    textSecondary: Color(0xFF646A73),
    textTertiary: Color(0xFF8F959E),
    divider: Color(0xFFE5E6EB),
    border: Color(0xFFD0D3D9),
    success: Color(0xFF00A870),
    warning: Color(0xFFFF9F0A),
    danger: Color(0xFFF54A45),
    selectedBg: Color(0xFFE8F0FE),
    scrollbarThumb: Color(0xFFC1C1C1),
    windowBtnHover: Color(0xFFE5E6EB),
);

const kPaletteWarmGold = AppPalette(
    id: 'warmGold',
    name: '暖阳金 · 圣堂',
    primary: Color(0xFFC68A1B),
    primaryHover: Color(0xFFA8730F),
```

```
    accent: Color(0xFF5B8C5A),
    pageBg: Color(0xFFDFBF6),
    cardBg: Color(0xFFFFFFFF),
    titleBarBg: Color(0xFFFFFFFF),
    topBarBg: Color(0xFFBF6EA),
    sidebarBg: Color(0xFF6EFD),
    rightPanelBg: Color(0xFF8F2E5),
    versionBarBg: Color(0xFFF0E7CE),
    playBarBg: Color(0xFFCFAF4),
    statusBarBg: Color(0xFF8F4EA),
    controlBg: Color(0xFF7EFD),
    controlBorder: Color(0xFFA2947A),
    lyricsBg: Color(0xFFCF4E0),
    textPrimary: Color(0xFF3A3124),
    textSecondary: Color(0xFF6B6152),
    textTertiary: Color(0xFF97896F),
    divider: Color(0xFFE9DFC8),
    border: Color(0xFFD6C9AE),
    success: Color(0xFF5B8C5A),
    warning: Color(0xFFD97A06),
    danger: Color(0xFFC0392B),
    selectedBg: Color(0xFFF5E7C3),
    scrollbarThumb: Color(0xFFCBBF9F),
    windowBtnHover: Color(0xFFEFE6CF),
);
const kPaletteCalmGreen = AppPalette(
    id: 'calmGreen',
    name: '静谧绿 · 草木',
    primary: Color(0xFF2F7D5A),
    primaryHover: Color(0xFF25654A),
    accent: Color(0xFF5BA37E),
    pageBg: Color(0xFFBFFFC),
    cardBg: Color(0xFFFFFFFF),
    titleBarBg: Color(0xFFFFFFFF),
    topBarBg: Color(0xFF2F8F4),
    sidebarBg: Color(0xFFEBF4EE),
    rightPanelBg: Color(0xFFEFF7F2),
    versionBarBg: Color(0xFFE3F0E7),
    playBarBg: Color(0xFF7FBF8),
    statusBarBg: Color(0xFFF1F7F3),
    controlBg: Color(0xFFEAF4EE),
    controlBorder: Color(0xFF95A89D),
    lyricsBg: Color(0xFFEEF8F2),
    textPrimary: Color(0xFF22322B),
    textSecondary: Color(0xFF5C6E65),
    textTertiary: Color(0xFF8A9A92),
    divider: Color(0xFFDEE8E1),
    border: Color(0xFFC9D6CE),
    success: Color(0xFF3E8F6A),
    warning: Color(0xFFD9840A),
```

```
danger: Color(0xFFD04A44),
selectedBg: Color(0xFFE1F0E7),
scrollbarThumb: Color(0xFFB9C6BE),
windowBtnHover: Color(0xFFE2EDE6),
);
const kPaletteElegantPurple = AppPalette(
  id: 'elegantPurple',
  name: '典雅紫 · 暮云',
  primary: Color(0xFF6E56CF),
  primaryHover: Color(0xFF5B46B3),
  accent: Color(0xFF1FA8A0),
  pageBg: Color(0xFFFFBFAFD),
  cardBg: Color(0xFFFFFFFF),
  titleBarBg: Color(0xFFFFFFFF),
  topBarBg: Color(0xFFF4F1FB),
  sidebarBg: Color(0xFFE7F7F7),
  rightPanelBg: Color(0xFFF1EDF8),
  versionBarBg: Color(0xFFE5DFF4),
  playBarBg: Color(0xFFFAF8FD),
  statusBarBg: Color(0xFFF2EFF9),
  controlBg: Color(0xFFECE6F8),
  controlBorder: Color(0xFFA095BA),
  lyricsBg: Color(0xFFF1EBFA),
  textPrimary: Color(0xFF2E2A3C),
  textSecondary: Color(0xFF6A6480),
  textTertiary: Color(0xFF9790AC),
  divider: Color(0xFFE4DFEE),
  border: Color(0xFFCEC6DE),
  success: Color(0xFF17A094),
  warning: Color(0xFFE08B0C),
  danger: Color(0xFFD34D55),
  selectedBg: Color(0xFFECE5FA),
  scrollbarThumb: Color(0xFFC2BACD),
  windowBtnHover: Color(0xFFE9E3F2),
);
const kPaletteMidnightDark = AppPalette(
  id: 'midnightDark',
  name: '暗夜墨 · 深色',
  brightness: Brightness.dark,
  primary: Color(0xFF4A8DF7),
  primaryHover: Color(0xFF3B75D4),
  accent: Color(0xFF22C3A6),
  pageBg: Color(0xFF171A21),
  cardBg: Color(0xFF1F232C),
  titleBarBg: Color(0xFF1E232D),
  topBarBg: Color(0xFF242B37),
  sidebarBg: Color(0xFF191D26),
  rightPanelBg: Color(0xFF1C212B),
  versionBarBg: Color(0xFF29303D),
  playBarBg: Color(0xFF1E232C),
```

```

    statusBarBg: Color(0xFF252B36),
    controlBg: Color(0xFF262C38),
    controlBorder: Color(0xFF3E4654),
    lyricsBg: Color(0xFF20252F),
    textPrimary: Color(0xFFE8EAED),
    textSecondary: Color(0xFFA9B0BC),
    textTertiary: Color(0xFF7D8595),
    divider: Color(0xFF2C313C),
    border: Color(0xFF3A4150),
    success: Color(0xFF2FC38A),
    warning: Color(0xFFE8A33D),
    danger: Color(0xFFE0564F),
    selectedBg: Color(0xFF26344A),
    scrollbarThumb: Color(0xFF4A5260),
    windowBtnHover: Color(0xFF2C313C),
  );
  const List<AppPalette> kThemes = [
    kPaletteMorning,
    kPaletteWarmGold,
    kPaletteCalmGreen,
    kPaletteElegantPurple,
    kPaletteMidnightDark,
  ];
  AppPalette themeById(String? id) {
    return kThemes.firstWhere((t) => t.id == id, orElse: () => kThemes.first);
  }
  class ThemeController {
    ThemeController._();
    static final ThemeController instance = ThemeController._();
    final ValueNotifier<AppPalette> notifier = ValueNotifier(kPaletteMorning);
    AppPalette get current => notifier.value;
    void switchTo(AppPalette palette) {
      if (palette.id == current.id) return;
      notifier.value = palette;
    }
  }
}
import 'dart:convert';
class Hymn {
  final int id;
  final String hymnNumber;
  final String title;
  final String lyricist;
  final String composer;
  final String sourceInfo;
  final int verseCount;
  final List<String> verses;
  final String staffImagePath;
  final String numberedImagePath;
  final String staffPngPath;
  final String numberedPngPath;

```

```
final Map<String, String> audioVersions;
final List<String> audioVersionList;
final String downloadStatus;
final String integrityStatus;
const Hymn({
  required this.id,
  required this.hymnNumber,
  required this.title,
  required this.lyricist,
  required this.composer,
  required this.sourceInfo,
  required this.verseCount,
  required this.verses,
  required this.staffImgPath,
  required this.numberedImgPath,
  required this.staffPngPath,
  required this.numberedPngPath,
  required this.audioVersions,
  required this.audioVersionList,
  required this.downloadStatus,
  required this.integrityStatus,
});
String get number => hymnNumber;
String get author => lyricist;
String get category => '';
String get audio => '';
String get nameWithNumber => '$hymnNumber · $title';
String get statusMeta =>
  '第 $hymnNumber 首 · $title · 词: $lyricist · 曲: $composer';
static Map<String, String> parseAudioVersions(String raw) {
  if (raw.isEmpty) return const {};
  try {
    final decoded = jsonDecode(raw);
    if (decoded is Map<String, dynamic>) {
      return decoded.map((k, v) => MapEntry(k, v.toString()));
    }
  } catch (_) {}
  return const {};
}
static List<String> parseAudioVersionList(String raw) {
  if (raw.isEmpty) return const [];
  try {
    final decoded = jsonDecode(raw);
    if (decoded is List) {
      return decoded.map((e) => e.toString()).toList();
    }
  } catch (_) {}
  return const [];
}
String? audioPathOf(String version) => audioVersions[version];
```

```

factory Hymn.fromDbRow(Map<String, Object?> row) {
  String str(String key) => (row[key] as String?) ?? '';
  int intVal(String key) => (row[key] as int?) ?? 0;
  final verses = <String>[
    for (var i = 1; i <= 10; i++) str('verse_$i'),
  ].where((v) => v.trim().isEmpty).toList();
  return Hymn(
    id: intVal('id'),
    hymnNumber: str('hymn_number'),
    title: str('title'),
    lyricist: str('lyricist'),
    composer: str('composer'),
    sourceInfo: str('source_info'),
    verseCount: intVal('verse_count'),
    verses: verses,
    staffImagePath: str('staff_img_path'),
    numberedImagePath: str('numbered_img_path'),
    staffPngPath: str('staff_png_path'),
    numberedPngPath: str('numbered_png_path'),
    audioVersions: parseAudioVersions(str('audio_versions')),
    audioVersionList: parseAudioVersionList(str('audio_version_list')),
    downloadStatus: str('download_status'),
    integrityStatus: str('integrity_status'),
  );
}
}
import 'dart:convert';
class HymnCategory {
  final int id;
  final String category;
  final String subcategory;
  final List<MapEntry<String, int>> hymns;
  const HymnCategory({
    required this.id,
    required this.category,
    required this.subcategory,
    required this.hymns,
  });
  factory HymnCategory.fromDbRow(Map<String, Object?> row) {
    String str(String key) => (row[key] as String?) ?? '';
    final hymns = <MapEntry<String, int>>[];
    try {
      final decoded = jsonDecode(str('hymns'));
      if (decoded is List) {
        for (final item in decoded) {
          if (item is Map<String, dynamic>) {
            item.forEach((k, v) => hymns.add(MapEntry(k, (v as num).toInt())));
          }
        }
      }
    }
  }
}

```

```

    } catch (_) {}
    return HymnCategory(
      id: (row['id'] as int?) ?? 0,
      category: str('category'),
      subcategory: str('subcategory'),
      hymns: hymns,
    );
  }
}
import 'dart:convert';
class Playlist {
  final int id;
  final String name;
  final String createdAt;
  final String updatedAt;
  final List<MapEntry<String, int>> hymns;
  const Playlist({
    required this.id,
    required this.name,
    required this.createdAt,
    required this.updatedAt,
    required this.hymns,
  });
  int get count => hymns.length;
  factory Playlist.fromDbRow(Map<String, Object?> row) {
    String str(String key) => (row[key] as String?) ?? '';
    final hymns = <MapEntry<String, int>>[];
    try {
      final decoded = jsonDecode(str('hymns'));
      if (decoded is List) {
        for (final item in decoded) {
          if (item is Map<String, dynamic>) {
            item.forEach((k, v) => hymns.add(MapEntry(k, (v as num).toInt())));
          }
        }
      }
    } catch (_) {}
    return Playlist(
      id: (row['id'] as int?) ?? 0,
      name: str('name'),
      createdAt: str('created_at'),
      updatedAt: str('updated_at'),
      hymns: hymns,
    );
  }
  static String hymnsToJson(List<MapEntry<String, int>> hymns) {
    return jsonEncode(
      hymns.map((e) => {e.key: e.value}).toList(growable: false));
  }
}

```